Reinforcement Learning With Multiple Relational Attention for Solving Vehicle Routing Problems

Yunqiu Xu, *Graduate Student Member, IEEE*, Meng Fang, Ling Chen, *Senior Member, IEEE*, Gangyan Xu, *Member, IEEE*, Yali Du, and Chengqi Zhang, *Senior Member, IEEE*

***Abstract*—In this article, we study the reinforcement learning (RL) for vehicle routing problems (VRPs). Recent works have shown that attention-based RL models outperform recurrent neural network-based methods on these problems in terms of both effectiveness and efficiency. However, existing RL models simply aggregate node embeddings to generate the context embedding without taking into account the dynamic network structures, making them incapable of modeling the state transition and action selection dynamics. In this work, we develop a new attention-based RL model that provides enhanced node embeddings via *batch normalization reordering* and *gate aggregation*, as well as *dynamic-aware context embedding* through an attentive aggregation module on multiple relational structures. We conduct experiments on five types of VRPs: 1) travelling salesman problem (TSP); 2) capacitated VRP (CVRP); 3) split delivery VRP (SDVRP); 4) orienteering problem (OP); and 5) prize collecting TSP (PCTSP). The results show that our model not only outperforms the learning-based baselines but also solves the problems much faster than the traditional baselines. In addition, our model shows improved generalizability when being evaluated in large-scale problems, as well as problems with different data distributions.**

***Index Terms*—Attention mechanism, combinatorial optimization, deep reinforcement learning (DRL), traveling salesman problem, vehicle routing problem (VRP).**

I. Introduction

VEHICLE routing problems (VRPs) are a typical class of combinatorial optimization problems that have been studied for several decades in both operations research and computer science [1]. These problems are fundamental in many industrial areas, such as transportation, logistics, communication networks, and product and service systems. Given a set of customer nodes to be visited and specified constraints (e.g., capacity and time) to be satisfied, the goal of a VRP can be briefly described as to find the optimal route (or route sets) with minimal total cost. It is known as an NP-hard problem and is intractable to find the optimal solution via an exhaustive search even with only a few hundred nodes. Many effective solutions have been proposed that can be classified into two groups: 1) exact solvers, such as [2]–[4], which are guaranteed to find the optimal solution by sacrificing computing time and 2) approximate solvers, such as [5]–[7], which customize heuristics to make a tradeoff between the optimality and computational efficiency. However, these methods require prior knowledge and handcrafted rules that cannot be adapted to varied and dynamic routing scenarios, greatly limiting their applications. In recent years, the evolutionary algorithms have been proven to be suitable to replace carefully designed heuristics [8]–[11]. However, according to [12], they are inflexible as these algorithms have to be reinitialized to cater for even slight changes of the tasks (e.g., changing the coordinates of a node). Besides, to obtain near-optimal solutions, these algorithms usually require a large population and a large number of iterations, leading to a long running time.

With the prosperity of deep learning (DL) and deep reinforcement learning (DRL), it is promising to solve VRP via neural network approximation [13]. As a result, the solving speed can be boosted by GPU computing in batch, and the dependency on expert knowledge can be reduced by interacting with data. A representative example is pointer networks (PtrNet) [14] that utilizes a sequence-to-sequence model with an attention mechanism to solve the travelling salesman problem (TSP). Due to the difficulty of obtaining a sufficient amount of perfect labels for supervised learning, especially for large-scale problems, reinforcement learning (RL) becomes more attractive as RL agents can be trained to directly optimize the task objective. Similar to PtrNet, typical RL solvers [15]–[17] follow the encoder–decoder framework, where the encoder processes all the input nodes to obtain node embeddings at the beginning of each episode. Then, at each time step, the decoder updates the context embedding and selects the next node to visit based on the attention value. Compared with the recurrent neural network (RNN)-based architecture used in PtrNet and its extensions in RL [15], [16], recent methods, such as the attention model (AM) [17], adopt the transformer-based architecture as well as the self-attention

Manuscript received 1 June 2020; revised 24 November 2020 and 30 March 2021; accepted 28 May 2021. Date of publication 8 July 2021; date of current version 16 September 2022. This work was supported in part by Australian Research Council under Grant DP180100966; in part by the National Natural Science Foundation of China under Grant 71804034; and in part by the Science and Technology Innovation Committee of Shenzhen under Grant JCYJ20180306171958907. This article was recommended by Associate Editor D. Zhao. *(Corresponding authors: Ling Chen; Gangyan Xu.)*

Yunqiu Xu, Ling Chen, and Chengqi Zhang are with the School of Computer Science, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: yunqiu.xu@student.uts.edu.au; ling.chen@uts.edu.au; chengqi.zhang@uts.edu.au).

Meng Fang is with the Department of Mathematics and Computer Science, Data Mining, Eindhoven University of Technology, 5612 AZ Eindhoven, The Netherlands (e-mail: m.fang@tue.nl).

Gangyan Xu is with the School of Architecture, Harbin Institute of Technology, Shenzhen 518055, China (e-mail: gagexgy@gmail.com).

Yali Du is with the Department of Computer Science, University College London, London WC1E 6BT, U.K. (e-mail: yali.du@ucl.ac.uk).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCYB.2021.3089179.

Digital Object Identifier 10.1109/TCYB.2021.3089179

2168-2267 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

mechanism [18] in both the encoder and the decoder, achieving high computational speed and significant performance gains.

For RL (especially DRL), building effective state representation is the first and most important step. However, the state representation generated by existing attention-based RL models cannot well reflect the state changes, which is essential in solving VRP. In many RL frameworks for VRP, the state representation consists of the node embeddings and the context embedding. While the node embeddings are encoded with static information, the context embedding should be able to capture the state transition dynamics and the effect of action selection. For RNN-based models [15], [16], the context embedding is the hidden context of the recurrent decoder, so that the transition dynamics is captured implicitly. For attention-based RL models such as AM [17], the context embedding is formulated as the combination of the graph embedding (i.e., the aggregation of node embeddings) and the embeddings of the first and the last selected nodes. While the last selected node is updated every time step, the graph embedding is fixed as the average of all node embeddings. Although this strategy is simple and the experimental results are promising, tracking only the first and the last selected nodes is insufficient to represent the state. Take TSP as an example. With the process of node selection, the nodes are divided into two subsets: 1) *visited* and 2) *unvisited*, which should be considered differently. However, this difference cannot be reflected by the current context embedding.

In this article, we aim to develop a new attention-based RL model with enhanced context embedding to boost routing performance. As the quality of context embedding depends heavily on the node embeddings, we will apply modifications to both the encoder and the decoder. Inspired by the recent progress on the transformer-based architecture [19], [20], we first modify the transformer block in the encoder with batch normalization (BN) reordering and *gate aggregation* to achieve faster convergence and better node embeddings, which, in turn, contributes to better context embedding. For the decoder, we design a new attentive aggregation module to build multiple relational structures, such as respective graph embeddings for visited and unvisited nodes, to make the context embedding explicitly aware of state changing. We evaluate our model on five types of VRPs with various volumes of nodes: 1) the TSP; 2) the capacitated vehicle routing problem (CVRP); 3) the split delivery vehicle routing problem (SDVRP); 4) the orienteering problem (OP); and 5) the prize collecting TSP (PCTSP). The results show that our model achieves better performance than learning-based baselines, while enjoying high running speed compared to traditional baselines. Moreover, our model shows improved generalizability on solving large-scale problems [17] as well as problems with different data distributions. Our main contributions are summarized as follows.

1) We propose a new attention-based RL model for VRPs, which is more powerful with dynamic-aware context embedding.
2) We show that the dynamics of state transition and action selection can be captured via explicitly building multiple relational structures.
3) We show that the proposed model is capable of different types of VRPs. Besides, our model demonstrates improved generalizability in large-scale problems, and problems with different data distributions.

The remainder of this article is organized as follows. Section II reviews related work. Section III details our proposed DRL framework. The experiments and results are presented in Sections IV and V, respectively. Finally, Section VI concludes this article.

II. Related Work

A. Traditional Solvers for VRP

The VRPs have been extensively studied for decades and many solvers have been proposed [1], [21]. The traditional solvers can be divided as the exact solvers and approximate solvers. The exact solvers, such as Branch-and-Bound (BnB) [2], branch-cut-and-price (BCP) [3], and Concorde [4], are representative solvers specialized for TSP, which are guaranteed to find the optimal solution. However, these solvers are infeasible to solve large-scale problems due to high computational complexity. By carefully designing the heuristics, the search space and complexity can be reduced to make large-scale problems tractable. Representative heuristics include minimum-spanning tree with minimum-weight perfect matching [22], local search [5], Lin–Kernighan–Helsgaun (LKH) heuristic [6], memetic search [23], and harmony search [24]. There are also generic solvers, such as Google OR-tools [25] and Gurobi [26], which are capable for multiple kinds of combinatorial optimization problems. While traditional solvers require handcrafted engineering to design the heuristics, we aim to learn the solver by interacting with the data, mitigating the need for prior knowledge. The evolutionary algorithms (e.g., [8]–[11], [27], [28]) can also be used to replace the heuristics. However, these algorithms have to be reinitialized even though there are slight changes to the tasks, limiting their flexibility and adaptability.

B. DL, RL-Based Solvers for VRP

Deep RL has been used in many real-world problems, including games [96], [53], robotics [93]–[95], natural language processing [92], [89], and so on. Recently, as a promising direction it has been explored for traffic. By formulating the VRP as a sequential decision making process, there have been works applying tabular RL algorithms [29]–[31]. Typically, the RL algorithms can be combined with genetic algorithms to design hybrid metasearch heuristics [32]–[35]. For example, the Ant-Q algorithm [36] has been developed by combining the Q-learning algorithm with the ant colony optimization (ACO) [27] for solving TSP. The system can be further extended with multiple agents to handle problems with multiple routes, or multiple vehicles [37], [38]. Different from these efforts, where RL has to be combined with other methods to obtain the final solution, we aim to design RL-based solvers to find the solution individually. Besides, we focus on deep RL techniques to investigate how deep neural networks help RL solvers to solve the VRP.

The first attempt that combines neural networks with combinatorial optimization applied the Hopfield network for

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

small TSP instances [39]. With the advances of DL techniques that enable high-speed and large-scale computation via GPUs, recent works seek to design deep neural-network-based solvers. The PtrNet [14] applied a sequence-to-sequence learning framework [40] to TSP. It first encodes the nodes as embeddings, and then computes the attention values for individual nodes to select the next node to visit. By setting the reward as the negative total tour length, the neural combinatorial optimization model (NCO) extended PtrNet to RL, so that there is no need to precollect optimal solutions as labels [15]. Going beyond TSP, such encoder–decoder frameworks have been applied to other problems, such as the CVRP [16], [41], [42], covering salesman problems (CSPs) [43], TSP with time windows (TSP-TWs) [44], TSP-TW with Rejections [45], and problems with multiple objectives [12], [46] and multiple agents [47], [48]. Instead of focusing on a specific type of VRP, we aim to design a generic framework capable of different types of VRPs. Some works proposed enhancing techniques, such as the stepwise learning [41], memory module [42], and local search strategy [49]. Our work is complementary to them, as these techniques can be integrated with our framework straightforwardly.

Some recent works [17], [50], [51] have applied the transformer-based architecture [18], which has shown to outperform RNNs in a wide range of sequential processing tasks [20], [52], [53]. The work [51] focused on the multiple TSP (mTSP), where a model is designed to obtain a fractional solution, and then, the beam search is applied to obtain a feasible solution. In [50], the recurrent encoder of PtrNet's framework is replaced with a transformer-based encoder, and then, the RL model is combined with 2-opt heuristics. The AM [17] also adopted a transformer-based encoder. In addition, the self-attention mechanism is applied during decoding, and a greedy rollout baseline estimator is proposed to facilitate training. These modifications lead to AM's high performance, even though the context embedding is considered almost statically with handcrafted state updating strategies (e.g., tracking the last selected nodes). The AM framework also enjoys high flexibility since it can be applied to handle multiple types of problems besides TSP and CVRP. Similar to these works, we also use the transformer-based encoder, and we intend to follow AM to design a generic framework capable of different types of VRPs. However, instead of applying the vanilla transformer, we will devise strategies to obtain better node embeddings during encoding. Moreover, we focus on designing attentive aggregation modules to build dynamic-aware context embedding during decoding. The experiment in Section V-B shows that our developed strategies in both encoder and decoder account for performance improvement.

Since the node set can be formulated as a graph, some work adopted graph embedding [54] and graph neural networks (GNNs) [44], [55]–[57]. In [55] and [56], the GNNs are applied in a supervised manner to solve the TSP. In terms of RL, S2V-DQN is proposed in [54] to obtain graph embedding from nodes, and learn the tour construction rule via 1-step DQN [58], [59]. The graph PtrNets (GPNs) [44] improved generalizability for large-scale TSP by integrating GNN into the PtrNet's decoder of a hierarchical policy, while the computational speed is severely slowed down. Our model is also related to the graph representation learning. In our work, the transformer-based encoder without positional encoding can be treated as graph attention networks (GATs) [60]. Besides the entire node set, we also treat the node subsets as different graph structures and build graph representations upon them. Our model is flexible to be combined with the recent progress of GNNs (see [61]).

In addition to solving VRP with the encoder–decoder framework, some studies aim at combining DRL with heuristics-based methods, such as picking heuristics [62], [63] as the action and learning new heuristics [64]–[67]. While we focus on improving attention-based RL models such as AM, our model can also be used as a component for these works to provide better state representation (e.g., [63]).

III. Our Proposed Methodology

A. General Framework

We define the problem with respect to TSP, while the input can be slightly adjusted for other VRPs. The input is a set (or order-invariant sequence) of N nodes $s = \{x_1, x_2, \ldots, x_N\} \in \mathbb{R}^2$ where each node consists of 2-D coordinates. The output a is a permutation of nodes $(a_1, a_2, \ldots, a_N)$, where a_i denotes the index of the ith node. Starting from an arbitrary depot node x_{a_1}, the objective of the "salesman" is to visit each node exactly once (except the depot node, which will be returned to at the last step) while minimizing the total tour length

$$L(a|s) = \left\|x_{a_N} - x_{a_1}\right\|_2 + \sum_{i=1}^{N-1} \left\|x_{a_i} - x_{a_{i+1}}\right\|_2. \tag{1}$$

Similar to previous work [15]–[17], we formulate the task as an RL problem. Fig. 1 shows the general framework. Given a problem instance as the input, an episode is defined as an encoding–decoding process to obtain the solution, that is, the node permutation a. Before an episode, the nodes will be first encoded as node embeddings, which will be fixed throughout the episode. The decoding is an N-timestep process: for each time step t, the agent selects an unvisited node and the action a_t denotes the node index. The state s_t is defined as the node set s with transition dynamics information (e.g., node visitation). The reward is set as $-L(a|s)$ and will be assigned episodically. Given s, the learning objective is to learn a policy to obtain the permutation a with the maximum episodic reward $-L(a|s)$. The policy is parameterized by θ and factorized as

$$p_\theta(a|s) = \prod_{t=1}^{N} p_\theta(a_t|s_t, a_{1:t-1}). \tag{2}$$

B. Multiple Relational Attention Model

We build upon AM [17] and design our model with a multiple relational attention mechanism to better capture the transition dynamics. We denote our model as multiple relational AM (MRAM). Fig. 2 compares the architecture of the backbone AM [Fig. 2(a)] and the proposed MRAM [Fig. 2(b)]. Our MRAM is similar to AM in three aspects: 1) the transformer-based encoder; 2) the self-attention module in the decoder ("Attn"); and 3) the training strategy. The

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

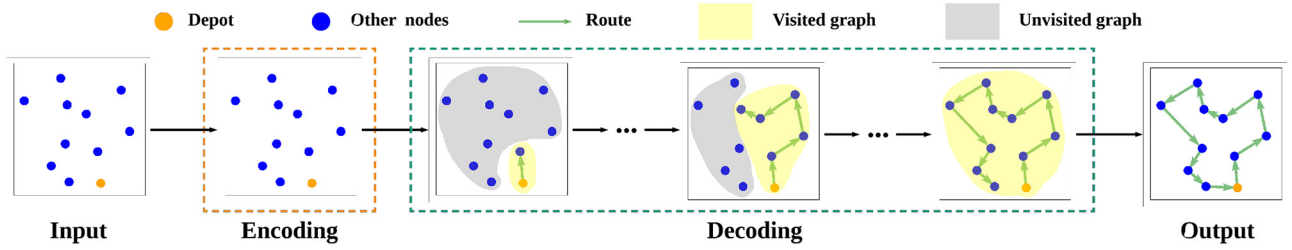


Fig. 1. RL framework for VRPs (TSP as an example). Before each episode, the nodes will be first encoded as node embeddings, and kept fixed throughout the episode. For each time step during the decoding process, an unvisited node will be selected. The final output is a permutation of nodes, denoting the route. In this work, we aim to capture the transition dynamics by modeling the visited and unvisited nodes as subgraphs (the shadowed parts).

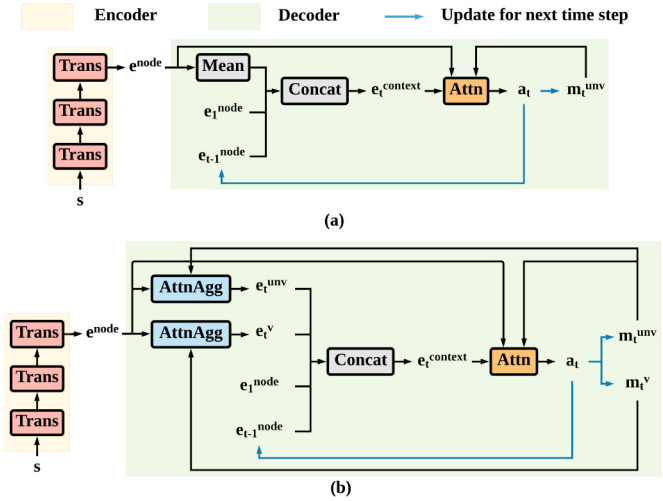


Fig. 2. (a) Backbone model AM. (b) Proposed model MRAM. Our major modifications lie in the transformer blocks in the encoder ("Trans"), and the attentive aggregation module in the decoder ("AttnAgg"). The blue arrowed lines denote that the information is updated for the next time step.

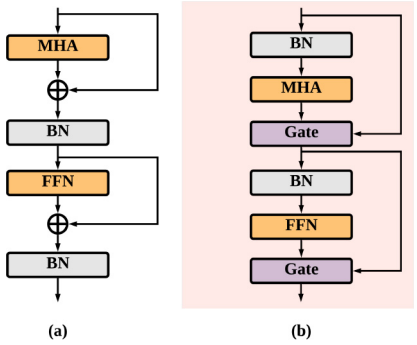


Fig. 3. Vanilla transformer block (a) and the modified block (b) in the encoder. We propose two modifications: 1) BN reordering (ReBN) to reorder the BN to be within the residual connection and 2) gate aggregation (Gate) to replace the additive aggregation with a gate layer.

encoder based on the transformer architecture [18] consists of three blocks ("Trans"). The positional encoding operation is removed since the input is order invariant. For each episode, the node set s will be first encoded as node embeddings $\boldsymbol{e}^{\text{node}}$. Then, the decoder will execute N times to obtain the permutation a. The state $\boldsymbol{s}_t$ is the combination of $\boldsymbol{e}^{\text{node}}$ and current context embedding $\boldsymbol{e}_t^{\text{context}}$, which is built from $\boldsymbol{e}^{\text{node}}$ and updated dynamically per time step. Given $\boldsymbol{e}^{\text{node}}$ and $\boldsymbol{e}_t^{\text{context}}$, the attention value (can be considered as a probability) for each node is then computed for action selection.

To obtain good $\boldsymbol{e}_t^{\text{context}}$ to capture the state transition dynamics, our MRAM makes two branches of modifications: 1) we enhance the vanilla transformer blocks ("Trans") with *BN reordering* and *gate aggregation* for high-quality node embeddings during encoding, since $\boldsymbol{e}_t^{\text{context}}$ depends heavily on $\boldsymbol{e}^{\text{node}}$ and 2) we design an *attentive aggregation module* in the decoder ("AttnAgg") to construct two relational structures: 1) unvisited graph embedding and 2) visited graph embedding, to enhance $\boldsymbol{e}_t^{\text{context}}$ with dynamic information. In the following, we sequentially illustrate how to achieve Branch 1 in Section III-C; how to achieve Branch 2 as well as the rest of the decoding process in Section III-D; and how to train MRAM via policy gradient in Section III-E.

C. Enhanced Transformer Block

According to [18], each transformer block consists of the multihead attention (MHA), feedforward network (FFN), and layer normalization (LN). The AM [17] adopted the vanilla transformer block except that LN is replaced by the BN [68]. Fig. 3(a) shows the block used in AM. For simplicity, we regard it as the vanilla block hereafter. As shown in Fig. 3(b), we introduce two modifications: 1) the BN reordering (ReBN) and 2) the gate aggregation (Gate).

Inspired by the effectiveness of normalization reordering in language processing tasks [19], we introduce the similar modification to RL. Specifically, we incorporate the BN operation into the residual connection to form an identity map between the input and output of MHA (or FFN). For the lth block, the input $\boldsymbol{e}^{(l-1)} \in \mathbb{R}^{N\times D}$ denotes a vector of N node embeddings, where D is the node feature dimension. $\boldsymbol{e}^{(l-1)}$ can be either the initially embedded nodes or the output of the $(l-1)$th block. The input embedding will be first normalized, and then processed by the MHA to compute the MHA

$$\boldsymbol{e}_{\text{BN}}^{(l-1)} = \text{BN}(\boldsymbol{e}^{(l-1)}) \tag{3}$$

$$\boldsymbol{e}_{\text{MHA}}^{(l)} = \text{MHA}\left(\boldsymbol{v}_{\text{Q}} = \boldsymbol{e}_{\text{BN}}^{(l-1)}, \boldsymbol{v}_{\text{K}} = \boldsymbol{e}_{\text{BN}}^{(l-1)}, \boldsymbol{v}_{\text{V}} = \boldsymbol{e}_{\text{BN}}^{(l-1)}\right). \tag{4}$$

The MHA has three input vectors: 1) the query $\boldsymbol{v}_{\text{Q}}$; 2) the key $\boldsymbol{v}_{\text{K}}$; and 3) the value $\boldsymbol{v}_{\text{V}}$, which are all set to be $\boldsymbol{e}_{\text{BN}}^{(l-1)}$, as the self-attention mechanism is applied in the transformer.

The gate aggregation is borrowed from the similar operations in the highway network [69] and GTrXL [20]. While the former [69] mainly focused on the computer vision tasks, the

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

latter [20] is similar to us in terms of applying the transformer-based architecture as well as the gate-like aggregation to RL tasks. However, the transformer in GTrXL is used as a memory module, while we leverage it for state representation learning. We replace the additive operation with a gated output connection; therefore, the weight factor computed from the input stream $\boldsymbol{e}_{\text{in}}$ is assigned to the output stream $\boldsymbol{e}_{\text{out}}$

$$G(\boldsymbol{e}_{\text{in}}, \boldsymbol{e}_{\text{out}}) = \boldsymbol{e}_{\text{in}} + \sigma(W\boldsymbol{e}_{\text{in}} + b) \odot \boldsymbol{e}_{\text{out}} \tag{5}$$

where W and b are learnable variables, and σ is the Sigmoid function. The output of the MHA is then aggregated as

$$\boldsymbol{e}^{(l)}_{\text{MHA}} = G\Big(\boldsymbol{e}^{(l-1)}, \boldsymbol{e}^{(l)}_{\text{MHA}}\Big). \tag{6}$$

The operations in the FNN are similar. The input $\boldsymbol{e}^{(l)}_{\text{MHA}}$ will be batchwisely normalized first and processed by fully connected layers. The output $\boldsymbol{e}^{(l)}$ is obtained via gate aggregation

$$\boldsymbol{e}^{(l)} = G\Big(\boldsymbol{e}^{(l)}_{\text{MHA}}, \text{FFN}\Big(\text{BN}\Big(\boldsymbol{e}^{(l)}_{\text{MHA}}\Big)\Big)\Big). \tag{7}$$

From the perspective of graph learning, the encoder is similar to the GATs [60]: the node set is considered as a fully connected graph and the embedding of each node is computed based on itself and all other nodes. The output of the last block is treated as $\boldsymbol{e}^{\text{node}} \in R^{N\times D}$ and will be fixed as static information throughout the episode.

D. Attentive Aggregation Module via Multiple Relations

In attention-based RL models such as AM [17], the context embedding $\boldsymbol{e}^{\text{context}}_t$ consists of the graph embedding, the first selected node $\boldsymbol{e}^{\text{node}}_1$, and the last selected node $\boldsymbol{e}^{\text{node}}_{t-1}$. Since the graph embedding is computed statically as the average of all node embeddings, the only component changing over time is the last selected node, which is not sufficient to capture state transition dynamics. In this work, we aim to introduce more information for $\boldsymbol{e}^{\text{context}}_t$ to make it aware of transitional dynamics. Specifically, we consider the visitation information by designing an attentive aggregation module (AttnAgg) to model the subsets of visited and unvisited nodes as different relational structures. We treat these structures as graphs, and represent them as graph embeddings. As shown in Fig. 4, this module takes $\boldsymbol{e}^{\text{node}}$ and the mask (e.g., the visited mask $\boldsymbol{m}^v_t$ is a binary vector where the visited nodes are set as 1, and others 0) as the input. $\boldsymbol{e}^{\text{node}}$ will first be projected to compute attentive weights, which is similar to the gate operation in the encoder. Then, the weights, mask, as well as node embeddings will be multiplied to obtain the masked node embeddings $\widehat{\boldsymbol{e}^{\boldsymbol{v}}}$

$$\widehat{\boldsymbol{e}^{\boldsymbol{v}}} = \text{Softmax}\Big(W_{\text{m}}\boldsymbol{e}^{\text{node}}\Big) \odot \boldsymbol{m}^v_t \odot \boldsymbol{e}^{\text{node}} \tag{8}$$

where W_{m} is a learnable variable. In order to obtain the graph embedding, instead of using one aggregation method only (e.g., averaging in AM), we tend to aggregate the node embeddings in multiple ways, so that the graph embedding will capture different aggregation preference. We obtain inspiration from the readout operation, which is commonly used for graph pooling [70]: given the input embedding, two vectors are computed via mean and max operations, respectively, and then concatenated as the final embedding. We keep the max operation to make the graph aware of parts with higher attention values. As we do not apply graph pooling to remove connections, we replace the mean operation with sum to reduce the influence of those less important

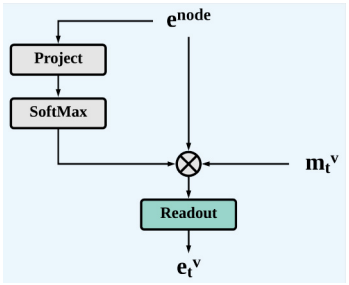


Fig. 4. Attentive aggregation module, where the node embeddings $\boldsymbol{e}^{\text{node}}$ are projected to compute attentive weights. Then, $\boldsymbol{e}^{\text{node}}$, the attentive weights as well as the visited mask $\boldsymbol{m}^v_t$ will be multiplied and aggregated by a modified readout layer to obtain visited graph embedding $\boldsymbol{e}^v_t$. For unvisited graph embedding, we use $\boldsymbol{e}^{\text{node}}$ and $\boldsymbol{m}^{\text{unv}}_t$ as inputs and obtain $\boldsymbol{e}^{\text{unv}}_t$.

$$\text{Readout}\Big(\widehat{\boldsymbol{e}^{\boldsymbol{v}}}\Big) = \text{Concat}\left(\sum_{i=1}^{N}\widehat{\boldsymbol{e}^{\boldsymbol{v}}}; \max_{i=1}^{N}\widehat{\boldsymbol{e}^{\boldsymbol{v}}}\right). \tag{9}$$

Similar to the visited graph embedding, we use the same mechanism for the unvisited nodes with different W_{m} and the unvisited mask $\boldsymbol{m}^{\text{unv}}_t$. That is, we build two different AttnAgg modules, considering that visited nodes and unvisited nodes have different semantic meanings and should be processed differently rather than being handled by a single module.

The visited graph embedding $\boldsymbol{e}^v_t$, the unvisited graph embedding $\boldsymbol{e}^{\text{unv}}_t$, and the node embedding of the first and the last visited nodes will be combined to form $\boldsymbol{e}^{\text{context}}_t$

$$\boldsymbol{e}^{\text{context}}_t = \text{Concat}\Big(\boldsymbol{e}^{\text{unv}}_t, \boldsymbol{e}^v_t, \boldsymbol{e}^{\text{node}}_1, \boldsymbol{e}^{\text{node}}_{t-1}\Big). \tag{10}$$

The state $\boldsymbol{s}_t$, which consists of both $\boldsymbol{e}^{\text{node}}$ and $\boldsymbol{e}^{\text{context}}_t$, are used to compute the attention values. Arbitrary methods can be used to compute attention values, such as the Bahdanau attention [71] used in PtrNet, or the self-attention [18] used in AM. We follow AM to obtain attention values in a manner similar to the self-attention mechanism. We first compute the MHA across $\boldsymbol{e}^{\text{context}}$ and $\boldsymbol{e}^{\text{node}}$. (We omit the subscript "t" for simplicity.) Different from (4), where the input vectors $\boldsymbol{v}_{\text{Q}}$, $\boldsymbol{v}_{\text{K}}$, and $\boldsymbol{v}_{\text{V}}$ are the same, we treat $\boldsymbol{e}^{\text{context}}$ as $\boldsymbol{v}_{\text{Q}}$, and let $\boldsymbol{e}^{\text{node}}$ be $\boldsymbol{v}_{\text{K}}$ and $\boldsymbol{v}_{\text{V}}$

$$\boldsymbol{e}^{\text{context}}_{\text{MHA}} = \text{MHA}\Big(\boldsymbol{v}_{\text{Q}} = \boldsymbol{e}^{\text{context}}, \boldsymbol{v}_{\text{K}} = \boldsymbol{e}^{\text{node}}, \boldsymbol{v}_{\text{V}} = \boldsymbol{e}^{\text{node}}\Big). \tag{11}$$

Then, for each node with index $i \in [1, N]$, we compute its attention $\boldsymbol{u}_i$ based on $\boldsymbol{e}^{\text{context}}_{\text{MHA}}$ and its node embedding $\boldsymbol{e}^{\text{node}}_i$, where W_{Q} and W_{K} are learnable. Similar to [15], we use tanh clipping to restrict the result within the range $[-C, C]$, where C is a constant. The unvisited mask $\boldsymbol{m}^{\text{unv}}$ is used here to prevent a visited node from being selected again

$$\boldsymbol{u}_i = \begin{cases} C \cdot \tanh\left(\dfrac{(W_{\text{Q}}\boldsymbol{e}^{\text{context}}_{\text{MHA}})^T(W_{\text{K}}\boldsymbol{e}^{\text{node}}_i)}{\sqrt{D}}\right), & \text{if } \boldsymbol{m}^{\text{unv}}_i = 1 \\ -\infty, & \text{otherwise.} \end{cases} \tag{12}$$

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Training via Policy Gradient

Input: number of epochs E, steps per epoch M, batch size B
Initialize: parameters θ

```
for epoch = 1, 2, ..., E do
    for step = 1, 2, ..., M do
        s_i ← GenerateInstance(), ∀i ∈ {1, 2, ..., B}
        a_i ← SampleSolution(s_i, p_θ), ∀i ∈ {1, 2, ..., B}
        Compute ∇L ← (1/B) Σ_{i=1}^{B} ∇L(θ|s_i) based on (15)
        θ ← Optimizer(θ, ∇L)
    end for
end for
```

The probabilities p_θ for action selection can then be obtained via softmax operation on the attention values of all nodes. The action a_t will be sampled or greedily selected based on p_θ. Finally, we update the visited and unvisited masks for the next time step. For each node with index $i \in [1, N]$, its visitation information is updated as

$$\boldsymbol{m}^v_{i,t+1} = \begin{cases} 1, & \text{if } i = a_t \\ \boldsymbol{m}^v_{i,t}, & \text{otherwise.} \end{cases} \tag{13}$$

$$\boldsymbol{m}^{\text{unv}}_{i,t+1} = 1 - \boldsymbol{m}^v_{i,t+1}. \tag{14}$$

E. Policy Gradient for Optimization

At the end of an episode, the solution a for the node set s is formed as the action selected per time step $\{a_1, a_2, \ldots, a_t, \ldots, a_N\}$. As the reward is assigned episodically, we optimize the model via policy gradient [72], [73]. The objective $\mathcal{L}(\theta|s)$ is the expected cost, which will be estimated with respect to the parameters θ

$$\nabla\mathcal{L}(\theta|s) = \mathbb{E}_{p_\theta(a|s)}\big[(L(a|s) - b(s))\nabla \log\ p_\theta(a|s)\big] \tag{15}$$

where $b(s)$ is the baseline estimator to reduce variance. According to previous work [15]–[17], the baseline estimator can be exponential, critic network, or rollout, which is a copy of the policy network with a lower updating frequency. Algorithm 1 shows the batchwise training process. The training will be conducted for E epochs, where each epoch consists of M updating steps. For each step, a batch of B training instances will be either generated on the fly, or sampled from a fixed training set. For each instance s_i within the current batch, the solution a_i will be obtained by sampling based on p_θ. The batchwise learning objective $\nabla\mathcal{L}$ is formulated as the average objective of each instance, and will be used for updating the parameters via an optimizer (e.g., the Adam optimizer [74]).

IV. Experiments

A. Experiment Setting

We conduct experiments on five types of VRPs: 1) TSP; 2) CVRP; 3) SDVRP; 4) PCTSP; and 5) OP. For all problems, we limit the number of vehicles as 1. As discussed before, the objective of TSP is to visit all nodes with the minimal tour length. The CVRP [1] is a generalized case of TSP, where each node consists of the coordinates and the demand, which will become zero if visited. The vehicle has limited capacity so that if it is fully loaded, it has to return to the depot node to unload before visiting new nodes. The SDVRP [75] is a generalized case of CVRP, by removing the restriction that each node can be visited only once. If a node's demand exceeds the vehicle's current capacity, the vehicle has to visit it multiple times. In the OP [76], each node is associated with the coordinates and an additional attribute prize. There is a maximum tour length constraint. The objective is to maximize the collected prize while not exceeding the tour length constraint. In particular, we consider the case of distance-based prize distribution, where a node's prize is proportional to its distance to the depot [77]. In the PCTSP [78], each node is associated with the coordinates and two attributes: 1) the prize and 2) the penalty. There is a minimum total prize constraint. The objective is to minimize the total tour length plus the total penalties of unvisited nodes, while collecting at least the prescribed minimum total prize. In particular, we consider the case of deterministic prize [17].

B. Baselines

We compare MRAM with both traditional and learning-based baselines. For TSP, we set up the following baselines.

1) *LKH [6]:* The state-of-the-art solver based on the LKH heuristic.
2) *Gurobi [26]:* A commercial optimization solver.
3) *Concorde [4]:* An exact solver specialized for TSP.
4) *OR-Tools [25]:* Google's operations research tools.
5) *Nearest Neighbor*: A solver that selects the nearest available neighbor of the last selected node.
6) *Nearest/Farthest/Random Insertion [5]:* Solvers that select the node based on the cheapest insertion cost.
7) *EliteGA [79]:* A genetic algorithm with elitist-preserving strategy.
8) *StudGA [80]:* A genetic algorithm, where the fittest individual shares genetic information with all others via simple operators.
9) *PtrNet-SL [14]:* PtrNets with supervised learning.
10) *PtrNet-RL [15]:* PtrNets with RL.
11) *RL-Greedy [16]:* An extension of PtrNet-RL, where the recurrent encoder is replaced with elementwise projecting to enable efficient information updating.
12) *EAN [50]:* An extension of PtrNet-RL with transformer-based encoder.
13) *GPN [44]:* An extension of PtrNet-RL with GNN-based decoder.
14) *AM [17]:* An attention-based RL model with transformer-based encoder and self-attention-based decoder.

For CVRP, we set two heuristic-based methods, LKH and OR-Tools, and two learning-based methods, RL-Greedy and AM, as baselines. For SDVRP, OP, and PCTSP, we set AM as the baseline. It is notable that for learning-based methods, better performance can be achieved via sampling strategies [15]–[17], [44] and ensemble learning [63]. However, we only consider greedy selection and single model in this work.

C. Implementation Details

Our implementation is based on AM's released code,[1] with the same hyperparameter settings. The node embedding

[1] https://github.com/wouterkool/attention-learn-to-route

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

TABLE I
POPULATION (POP.) AND MAXIMUM GENERATIONS (MAX GEN.) OF EVOLUTIONARY SOLVERS

Method	TSP20		TSP50		TSP100	
	Pop.	Max gen.	Pop.	Max gen.	Pop.	Max gen.
EliteGA	200	500	1000	2000	4000	5000
StudGA	400	1000	500	4000	3000	5000

dimension is 128. The encoder consists of three blocks, each with eight heads for MHA and 512 hidden dimension for FFN. In the decoder, the attention module consists of an MHA with eight heads followed by a single-head self-attention module. The tanh clip is applied with $C = 10$. For CVRP and SDVRP, we keep depot nodes unvisited for the entire episode since they will be revisited multiple times. For simplicity, we keep the semantic meaning of masks same in all problems: $\boldsymbol{m}_t^v$ and $\boldsymbol{m}_t^{\text{unv}}$ are binary masks so that they only consider the first-time-visitation of nodes.

We use the rollout baseline as the baseline estimator, which achieves better performance than other estimators [17]. The rollout baseline is a copy of policy network with frozen weights. The update frequency of the rollout baseline is per epoch and it will be updated to current policy only if the performance improvement is significant enough (over 5% on a separate dataset of 10 000 instances). Most of the traditional baselines are also provided by AM's released code except EliteGA and StudGA, which are implemented via Geatpy.[2] Table I shows the population and maximum generations of these evolutionary solvers.

We follow [16] and [17] to conduct experiments on problems with {20, 50, 100} nodes. For all problem instances, the 2-D node coordinates are sampled within the area $[0, 1] \times [0, 1]$. For CVRP and SDVRP, the capacity is set as {30, 40, 50}, respectively. The demand for each node is sampled from $\{1, 2, \ldots, 9\}$, and is normalized by the problem capacity. For OP, the maximum tour length constraint is set as {2, 3, 4}, respectively. For PCTSP, the minimum total prize constrain is set as 1, as the prizes can be normalized. Unless specified, the sampling process (e.g., for the node coordinates) follows uniform distribution by default.

We train the models (MRAM and the learning-based baselines) for 100 epochs. For each epoch, we generate 1 280 000 instances on the fly and train with batch size of 512 (except for CVRP100 and SDVRP100, where we generate 640 000 instances and train with batch size of 256). We use the Adam optimizer and set the learning rate as 10^{-4} without decay. We build a fixed test set with 1000 instances and evaluate the model with batch size of 100 after each training epoch. During testing, we use the greedy node selection strategy so that the node with the maximum attention value will be selected. For all problems, we evaluate the models on instances with the same number of nodes during training (e.g., training and testing on TSP20). Besides, for TSP and CVRP, we also evaluate the model generalizability on problems with a larger number of nodes (e.g., training on TSP20, and testing on TSP1000), and on problems with different distributions. Refer to Sections V-C and V-D for details. We evaluate the traditional baselines on 1000 testing instances (except for the evolutionary baselines on TSP100 with 100 instances). We run learning-based methods on one NVIDIA Tesla V100 PCIe card with 32-GB GPU memory, while the traditional methods are run on Intel Xeon E5-2687W with 20 cores.

[2]https://github.com/geatpy-dev/geatpy

D. Evaluation Metrics

Similar to [14]–[17], we measure the performance via the total tour length L and the optimality gap, which is defined as the normalized distance from the best result L_{best}

$$\text{Gap}_{\text{L}} = \frac{L - L_{\text{best}}}{L_{\text{best}}} \times 100\%. \tag{16}$$

For OP, we report the collected prize P and the optimality gap as follows:

$$\text{Gap}_{\text{P}} = \frac{P_{\text{best}} - P}{P_{\text{best}}} \times 100\%. \tag{17}$$

We also report the running time.

V. RESULTS AND DISCUSSIONS

A. Comparison With Baselines

Table II shows the major performance comparison, where the models are trained and tested on problems with the same number of nodes. In TSP and CVRP, the proposed MRAM outperforms other learning-based models. Its advantage becomes more significant with the increase of the node number. It shows the effectiveness of separately considering the visited nodes and unvisited nodes in large-scale problems, where tracking only the last node does not allow the agent to know the condition of the candidates (e.g., whether those near the last node have been visited or not). In addition, for small-scale problems such as TSP20, the performance of AM is already close to the optimal, making it very challenging to improve further. Comparing with CVRP, we are surprised to observe that the performance advantage of MRAM over AM is more significant in SDVRP. Although the relational structures only consider the first-time-visitation of nodes, it is still helpful in the case where each node can be visited for multiple times (SDVRP). In PCTSP and OP, MRAM outperforms AM in 5 out of 6 experiments, except OP100, where its performance is slightly worse than AM. There are two possible reasons for MRAM's worse performance: 1) the additional attributes, such as the prize and penalty, should be paid with more attention and 2) in PCTSP and OP, some nodes will never be visited throughout the episode. Therefore, it may be unsuitable to model the unvisited nodes. Overall, introducing relational structure helps to improve performance in most cases. In the following sections, we mainly focus on TSP and CVRP, which are best characterized by the division of visited and unvisited nodes. Nevertheless, our MARM is flexible to be designed with other forms of relational structures (e.g., graph structure with consideration of visitation counts or other attributes) to handle more task-specific cases, and we leave these extensions as future work.

In terms of the running time, due to additional computations in the encoder (gate aggregation) and decoder (projections in

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE OF MRAM VERSUS BASELINES. THOSE MARKED WITH (†) ARE FROM [14]–[16]. THE TSP100 RESULTS FOR ELITEGA AND STUDGA ARE OBTAINED FROM A TESTING SET OF 100 TSP100 INSTANCES. THE TSP100 RESULT FOR EAN IS OBTAINED BY MODEL TRAINED ON TSP50

Method	TSP20			TSP50			TSP100		
	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time
LKH	3.83	0.00%	5s	5.69	0.00%	27s	7.76	0.00%	96s
Gurobi	3.83	0.00%	2s	5.69	0.00%	12s	7.76	0.00%	77s
Concorde	3.83	0.00%	2s	5.69	0.00%	10s	7.76	0.00%	16s
OR-Tools†	3.85	0.52%	-	5.80	1.93%	-	7.99	2.96%	-
Nearest Neighbour	4.50	17.49%	<1s	6.99	22.85%	<1s	9.70	25.00%	<1s
Nearest Insertion	4.33	13.05%	<1s	6.78	19.16%	<1s	9.47	22.04%	<1s
Farthest Insertion	3.92	2.35%	<1s	6.00	5.45%	<1s	8.34	7.47%	1s
Random Insertion	4.00	4.44%	<1s	6.12	7.56%	<1s	8.51	9.66%	<1s
EliteGA	3.85	0.52%	30.4m	5.90	3.69%	9.7h	8.23	6.06%	-
StudGA	3.83	0.00%	2.0h	5.83	2.46%	9.2h	8.15	5.03%	-
PtrNet-SL†	3.88	1.31%	-	6.09	7.03%	-	-	-	-
PtrNet-RL†	3.89	1.57%	-	5.95	4.57%	-	8.30	6.96%	-
RL-Greedy†	3.97	3.66%	-	6.08	6.85%	-	8.44	8.76%	-
EAN†	3.86	0.78%	-	5.92	4.04%	-	8.42	8.50%	-
GPN	3.87	1.04%	0.49s	5.94	4.39%	1.35s	-	-	-
AM	3.84	0.26%	0.25s	5.79	1.76%	0.59s	8.10	4.38%	1.20s
MRAM (ours)	**3.84**	0.26%	0.37s	**5.76**	1.23%	0.91s	**8.05**	3.74%	2s

Method	CVRP20			CVRP50			CVRP100		
	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time
LKH	6.10	0.00%	310s	10.30	0.00%	1170s	15.54	0.00%	2234s
OR-Tools†	6.43	5.41%	-	11.31	9.80%	-	17.16	10.42%	-
RL-Greedy†	6.59	8.03%	-	11.39	10.58%	-	17.23	10.88%	-
AM	6.35	4.10%	0.61s	10.92	6.02%	1.36s	16.66	7.21%	2.50s
MRAM (ours)	**6.33**	3.77%	0.90s	**10.85**	5.34%	2.25s	**16.56**	6.56%	4.83s

Method	SDVRP20			SDVRP50			SDVRP100		
	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time
AM	6.38	0.63%	0.70s	10.88	0.65%	1.67s	16.78	2.13%	3.62s
MRAM (ours)	**6.34**	0.00%	0.96s	**10.81**	0.00%	2.38s	**16.43**	0.00%	-s

Method	PCTSP20			PCTSP50			PCTSP100		
	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time	Tour len.	Opt. gap	Time
AM	3.17	0.32%	0.24s	4.60	0.66%	0.60s	6.25	0.64%	1.09s
MRAM (ours)	**3.16**	0.00%	0.37s	**4.57**	0.00%	0.86s	**6.21**	0.00%	1.65s

Method	OP20			OP50			OP100		
	Prize.	Opt. gap	Time	Prize.	Opt. gap	Time	Prize.	Opt. gap	Time
AM	5.22	0.95%	0.31s	15.66	0.51%	0.88s	**31.53**	0.00%	1.25s
MRAM (ours)	**5.27**	0.00%	0.40s	**15.74**	0.00%	0.97s	31.52	0.03%	1.69s

attentive aggregation), our model is slowed down a bit compared to the original AM. However, it is still much faster than the traditional solvers such as LKH, which are not able to solve a batch of instances in parallel with GPU. When comparing with other learning-based algorithms, we find that our trade-off between performance and running speed is worthwhile, as AM is already a strong baseline with very high running speed. For example, as reported in [17], another model with the transformer-based architecture, EAN, is 150× slower than AM on 10 000 TSP50 instances, with inferior performance. In contrast, our MRAM consistently outperforms AM in most of the experiments with acceptable running time.

To study whether building relational structures is helpful to improve sample efficiency, which is still an open problem for DRL's applications, we compare the learning curve between MRAM and AM. A model is regarded as more sample efficient if it can reach equivalent performance of its competitors with fewer training instances or fewer training epochs, since the training instances are generated on the fly per epoch. As shown in Fig. 5, the optimality gap of both models keeps decreasing with the progress of training, showing that the performance can be further improved with more training epochs. The proposed MRAM achieves faster learning speed and higher sample efficiency than AM. Similar to Table II, its advantage becomes larger with the increase of the problem size. The dashed lines in TSP100 and CVRP100 show the final results achieved by the pretrained AM. In these two cases, MRAM requires only about 60 epochs to reach AM's performance.

To check the model's stability over different training and testing sets, we trained MRAM and AM with two random seeds (i.e., different training sets), and then evaluated them on three different testing sets. Table III shows the mean tour length as well as the standard deviation on each testing set. The performance of MRAM is stable as the obtained mean tour length is similar across different sets. This is because the building backbone AM itself is very stable, as shown in Table III. Another reason is that although the datasets are generated with different seeds, the instances follow the same data distribution, leading to small variance. Further investigation about the influence of data distribution could be found in Section V-D.

B. Ablation Study

To investigate the contribution of different components, we perform the ablation study on TSP50. Fig. 6(a) compares the influence of major modifications: "ModifiedEncoder" denotes modifications (i.e., BN reordering, gate aggregation) in the encoder, and "AttnAgg" denotes modifications (i.e.,

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

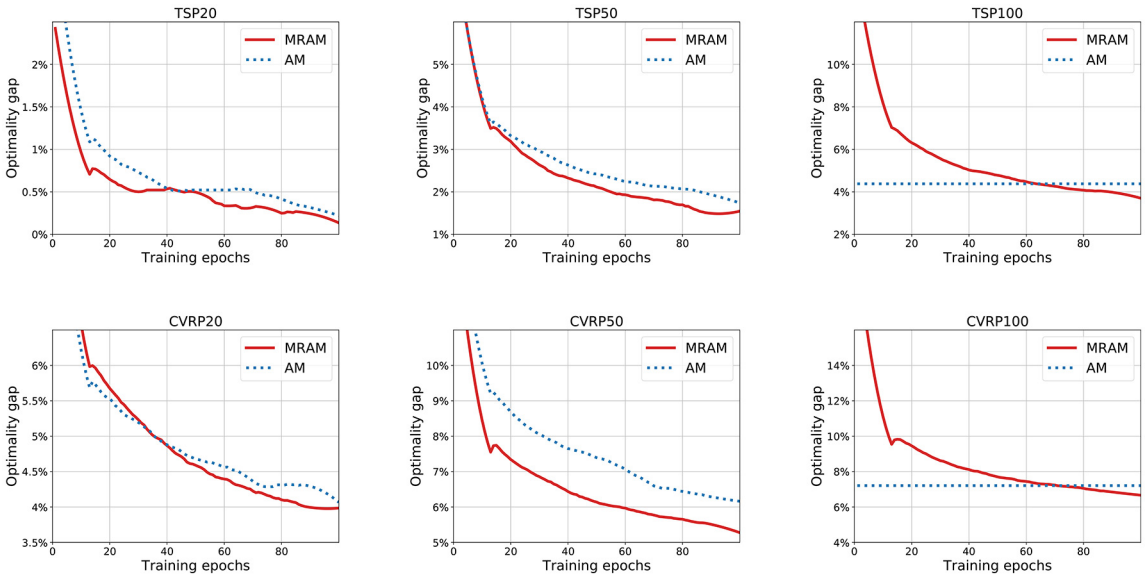


Fig. 5. Testing performance of MRAM and AM with respect to training epochs. The dashed lines in TSP100 and CVRP100 are results of pretrained models provided by [17]. Compared with AM, the optimality gap of MRAM drops faster, showing its higher sample efficiency.

TABLE III
TOUR LENGTH (MEAN ± STANDARD DEVIATION) OF TWO RANDOM SEEDS OVER THREE TESTING SETS

Method	TSP20			TSP50			TSP100		
	#1	#2	#3	#1	#2	#3	#1	#2	#3
AM,1234	3.84±0.01	3.84±0.01	3.84±0.01	5.79±0.01	5.79±0.01	5.79±0.01	8.10±0.01	8.10±0.01	8.11±0.01
AM,4321	3.84±0.01	3.84±0.01	3.84±0.01	5.80±0.01	5.79±0.01	5.79±0.01	8.10±0.01	8.11±0.01	8.11±0.01
MRAM,1234	3.84±0.01	3.84±0.01	3.84±0.01	5.76±0.01	5.76±0.01	5.77±0.01	8.05±0.01	8.05±0.01	8.05±0.01
MRAM,4321	3.84±0.01	3.83±0.01	3.84±0.01	5.77±0.01	5.76±0.01	5.76±0.01	8.05±0.01	8.05±0.01	8.07±0.01
Method	CVRP20			CVRP50			CVRP100		
	#1	#2	#3	#1	#2	#3	#1	#2	#3
AM,1234	6.35±0.03	6.37±0.04	6.35±0.03	10.92±0.05	10.94±0.05	10.95±0.05	16.66±0.06	16.64±0.06	16.69±0.06
AM,4321	6.36±0.03	6.36±0.03	6.35±0.03	10.92±0.05	10.94±0.05	10.92±0.05	16.67±0.06	16.64±0.06	16.66±0.06
MRAM,1234	6.33±0.03	6.34±0.03	6.34±0.03	10.85±0.04	10.85±0.04	10.87±0.04	16.56±0.06	16.57±0.06	16.57±0.06
MRAM,4321	6.35±0.03	6.33±0.03	6.33±0.03	10.85±0.04	10.85±0.04	10.86±0.04	16.56±0.06	16.56±0.06	16.58±0.06

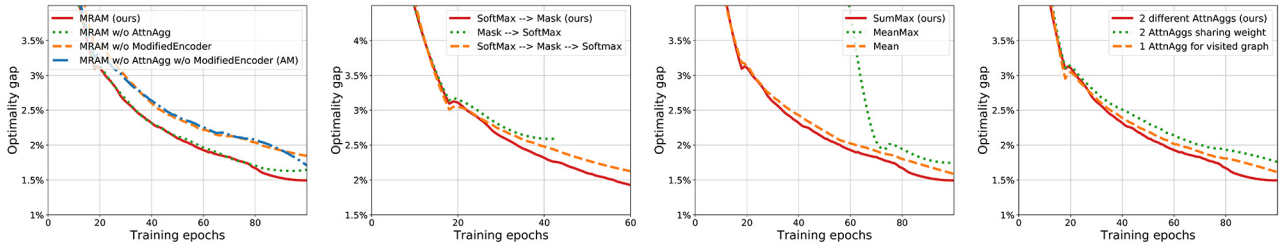


Fig. 6. Ablation study of (a) major modifications, where "w/o X" denotes that modification "X" is removed from MRAM, (b) order of operations in AttnAgg, (c) aggregation method in AttnAgg, and (d) number of AttnAggs. All the comparisons are based on TSP50.

attentive aggregation module) in the decoder. The employment of the modified encoder generates better node embeddings and speed up the convergence (e.g., "AM" versus "MRAM w/o AttnAgg"). The use of AttnAgg solely in the decoder makes learning unstable and cannot help to improve the performance (e.g., "AM" versus "MRAM w/o ModifiedEncoder"). However, when being used together with the modified encoder, the AttnAgg in the decoder further reduces the optimality gap to be less than 1.5% (e.g., "MRAM w/o AttnAgg" versus "MRAM"). Overall, the performance improvement of MRAM is a joint result of modifications in the encoder and decoder.

Fig. 6(b) compares the order of operations in AttnAgg modules. Compared with our model, where the mask is multiplied independently after computing attention values ("SoftMax → Mask"), another two variants with conditional attention values on the mask (i.e., "Mask → SoftMax" and "SoftMax → Mask → SoftMax") perform worse. This can be explained by the difference between soft and hard attention [81]. Computing attention values after masking makes the attention similar to soft attention, such that those masked nodes still have effects on the graph embedding. This is unfavorable in this work since we would like to consider visited nodes and unvisited nodes separately. In contrast, the operation order used in MRAM is

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
GENERALIZATION RESULTS ON TSP {20, 50, 100, 250, 500, 1000}

Method	TSP20	TSP50	TSP100	TSP250	TSP500	TSP1000
AM,TSP20	3.85	5.94	9.02	16.04	24.84	38.63
AM,TSP50	3.87	5.79	8.14	13.95	21.61	33.66
MRAM,TSP20	**3.84**	5.91	8.85	15.85	24.72	37.62
MRAM,TSP50	3.87	**5.76**	**8.12**	**13.77**	**21.25**	**33.03**

TABLE V
GENERALIZATION RESULTS ON CVRP {20, 50, 100}

Method	CVRP20	CVRP50	CVRP100
AM,CVRP20	6.35	11.46	18.65
AM,CVRP50	6.58	10.92	16.94
MRAM,CVRP20	**6.33**	11.46	18.42
MRAM,CVRP50	6.57	**10.85**	**16.86**

similar to hard attention, which prevents all irrelevant nodes from being built into graph embedding, leading to better performance.

Fig. 6(c) compares the aggregation methods in AttnAgg modules. The results show that the "SumMax" Readout used in MRAM helps to achieve better performance than averaging aggregation (i.e., "Mean"), showing the effectiveness of combining different aggregation methods. However, it is unexpected to observe that the use of original Readout ("MeanMax") leads to poor performance. This phenomenon may be attributed to the overfocusing on the max part. Take unvisited graph embedding as an example. The mean value of attentive node embeddings will become smaller with the reduction of unvisited nodes [according to (8) and (9), the values of visited nodes will be 0 while the number of nodes will not change]. While the max part will be less affected in this process, the graph embedding constructed via original readout ("MeanMax") may be focusing on the max part too much, which results in the performance even worse than using the "Mean" aggregation only.

Fig. 6(d) compares models with different numbers of AttnAgg modules. The result shows that using two different AttnAgg modules leads to better performance, since these two modules consider visited nodes and unvisited nodes with different attention preferences. In contrast, when we build two AttnAggs but share weights among them (i.e., "2 AttnAggs sharing weight"), the result is even worse than considering visited graph only (i.e., "1 AttnAgg for visited graph"). The variant "2 AttnAggs sharing weight" is similar to processing visited and unvisited graphs with a single module. Without considering whether a subgraph denotes the visited nodes or unvisited ones, the obtained graph embeddings as well as the context embedding will have misleading information, which affects decision making.

C. Generalization on Larger Scale Problems

For large-scale problems such as TSP1000, it is intractable to train the model from scratch, making generalizability an important issue. In this section, we analyze whether the modifications help to improve generalizability on these problems. To this end, we evaluate the trained models on testing sets with different number of nodes. Tables IV and V show the results on TSP and CVRP, respectively. The method "AM,TSP20" denotes that the model AM is trained on TSP20, and the

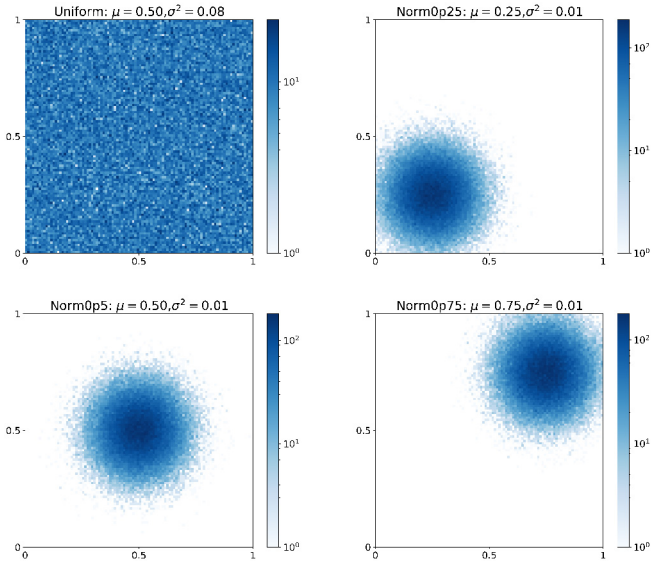


Fig. 7. Distribution of node coordinates.

columns denote the testing problems. By modeling different relational structures to make context embedding aware of the transition dynamics, our MRAM generalizes better than AM. We also find that while those trained and tested on the same problem setting achieve better performance than on different problem settings (e.g., "MRAM,TSP50" versus "MRAM,TSP20" on TSP20), the models trained on 50 nodes perform better in problems of a larger scale than those trained on 20 nodes (e.g., "MRAM,TSP50" versus "MRAM,TSP20" on TSP1000). Similar generalization results can be observed in the existing work for multiple-objective setting [12]. From the perspective of transfer learning [82], this performance improvement can be explained by the smaller gap between the source (training) and the target (testing) datasets. While adding more complex modules in the decoder will be beneficial to model the dynamics and thus, further improve the generalizability [44], we argue that this will lead to great speed drop since the decoder needs to execute N times. For learning-based methods, it is important to consider the structural complexity to tradeoff between the generalizability and running time.

D. Generalization on Different Distributions

As illustrated in Section IV-C, the instances of all problems are drawn from the uniform distribution $U[0, 1]$ by default. For learning-based methods, it is worthwhile to study the influence of data distribution on the model performance. In this section, we focus on varying the distribution of node coordinates, and investigating the model's generalizability on the testing sets with different distributions. Specifically, we denote the default distribution as "Uniform," and consider another three normal-like[3] distributions.

1) "Norm0p25," which follows distribution $N(0.25, 0.01)$ with mean $\mu = 0.25$ and variation $\sigma^2 = 0.01$.
2) "Norm0p5," which follows distribution $N(0.5, 0.01)$.
3) "Norm0p75," which follows distribution $N(0.75, 0.01)$.

[3]We still limit the generated nodes to be within the area $[0, 1] \times [0, 1]$.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

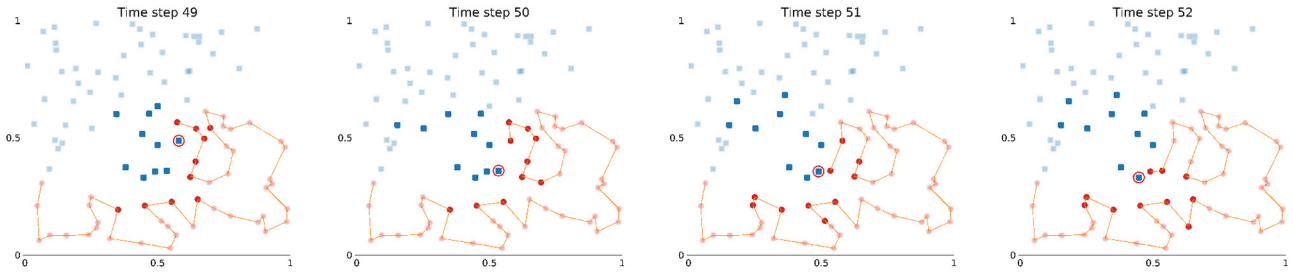


Fig. 8. Node selecting process of a TSP100 instance. The red filled circles are visited nodes, the blue squares are unvisited nodes, and the line denotes the route. Those with top ten highest attention values in each subset are highlighted with darker color. The chosen node to visit (action for this time step) is within the red circle.

TABLE VI
GENERALIZATION RESULTS ON TSP50 WITH DIFFERENT DISTRIBUTIONS

Method	Uniform	Norm0p25	Norm0p5	Norm0p75
AM,Uniform	5.79	2.81	2.74	2.85
AM,Norm0p5	6.33	2.47	2.44	2.46
MRAM,Uniform	**5.76**	2.78	2.72	2.76
MRAM,Norm0p5	6.34	**2.44**	**2.40**	**2.41**

TABLE VII
GENERALIZATION RESULTS ON CVRP50 WITH DIFFERENT DISTRIBUTIONS

Method	Uniform	Norm0p25	Norm0p5	Norm0p75
AM,Uniform	10.88	4.38	4.36	4.31
AM,Norm0p5	11.87	4.15	4.10	4.14
MRAM,Uniform	**10.81**	4.31	4.31	4.30
MRAM,Norm0p5	11.79	**4.11**	**4.07**	**4.12**

Fig. 7 visualizes the distributions of 100 000 sampled nodes. Tables VI and VII show the results on TSP50 and CVRP50, respectively. The method "AM,Uniform" denotes that the model AM is trained on a training set with uniform distribution, and the columns denote the distributions of different testing sets. Due to the small variation, the tour lengths of instances from normal-like distributions are much smaller than those from uniform distribution (e.g., "MRAM.Uniform" on "Uniform" and "Norm0p25"). Similar to the findings in Section V-C, when being trained on the same training set, the proposed MRAM generally performs better than the backbone model, even when the testing set follows a different distribution (e.g., "MRAM,Uniform" versus "AM,Uniform" on "Norm0p5"). We also observed that transferring the model between the normal-like distributions achieves better performance than transferring the model between a normal-like distribution and the uniform distribution (e.g., "MRAM,Uniform" versus "MRAM,Norm0p5" on "Norm0p75"), denoting that the normal-like distributions with different μ values have smaller domain gap. Combining the findings in Sections V-C and V-D, we conclude that MRAM improves generalizability over AM.

E. Qualitative Analysis

The attention mechanism has been used in RL to interpret the decision-making process [83], [84]. Inspired by this, we illustrate how modeling the relational structures helps to solve VRP by visualizing the attention values computed in the attentive aggregation modules. Fig. 8 shows the node selection process in a TSP100 instance. In each time step, the visited nodes are presented as red dots, the unvisited nodes are blue squares, and the line denotes the route. In the subsets of visited and unvisited nodes, we highlight the top ten nodes with highest attention values computed by the AttnAgg [e.g., $\text{Softmax}(W_{\text{m}}\boldsymbol{e}^{\text{node}})\odot\boldsymbol{m}_t^v$ in (8) for the subset of visited nodes], respectively. We observed that for TSP, the model learns to reduce the search space by paying more attention to those lying close to the boundary of visited and unvisited nodes, especially the nearest neighbors of the last selected node. Although there are successive operations (e.g., the readout operation, the attention computation between $\boldsymbol{e}^{\text{node}}$ and $\boldsymbol{e}_t^{\text{context}}$) before the final action selection, the node chosen to visit is included in the highlighted unvisited nodes, showing the effectiveness of AttnAgg.

VI. CONCLUSION

In this article, we investigated the attention-based RL framework for VRPs. We focused on designing a new context embedding that could better model transition dynamics and make RL more efficient. To this end, we first introduced a transformer-based encoder featured with BN reordering and gating layers to provide high quality node embeddings, which, in turn, improve the quality of context embedding. Then, we designed an attentive aggregation module to build dynamic-aware context embedding with multiple relational structures, which better reflect the state transition. The experimental results on five types of VRPs showed that our model not only outperforms the learning-based baselines but also solves the problems much faster than the traditional solvers. In addition, our model shows improved generalizability when applied to problems of a larger scale and/or with different data distributions.

With respect to future studies, we notice that there is still optimality gap between MRAM and the state-of-the-art heuristic-based methods. We will further improve the performance of MRAM, or combine MRAM with heuristic-based solvers, sampling strategies and ensemble learning methods. Moreover, considering the first-time-visitation is not the optimal choice for all problems, we will enhance model's flexibility to accommodate other forms of relational structures. We will also try to improve MRAM's generalizability to problems of even larger scales and/or with unseen distributions. The recent progress in solving larger

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

TSP [56], [85], [86], few-shot robotics tasks [87], procedurally generated games [88], and knowledge graph-enhanced reasoning [89] provides insights. Last but not least, we are interested in applying MRAM to practical problems [57], [90], [91].

REFERENCES

[1] P. Toth, D. Vigo, P. Toth, and D. Vigo, *Vehicle Routing: Problems, Methods, and Applications*. Philadelphia, PA, USA: SIAM, 2014.

[2] G. Dantzig, R. Fulkerson, and S. Johnson, "Solution of a large-scale traveling-salesman problem," *J. Oper. Res. Soc. America*, vol. 2, no. 4, pp. 393–410, 1954.

[3] M. Savelsbergh, "A branch-and-price algorithm for the generalized assignment problem," *Oper. Res.*, vol. 45, no. 6, pp. 831–841, 1997.

[4] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ, USA: Princeton Univ. Press, 2007.

[5] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "An analysis of several heuristics for the traveling salesman problem," *SIAM J. Comput.*, vol. 6, no. 3, pp. 563–581, 1977.

[6] K. Helsgaun, "An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems," Dept. People Technol., Roskilde Universitet, Roskilde, Denmark, Rep., 2017. [Online]. Available: https://forskning.ruc.dk/en/publications/an-extension-of-the-lin-kernighan-helsgaun-tsp-solver-for-constra

[7] X. Cai *et al.*, "The collaborative local search based on dynamic-constrained decomposition with grids for combinatorial multiobjective optimization," *IEEE Trans. Cybern.*, vol. 51, no. 5, pp. 2639–2650, May 2021.

[8] J. Wang, T. Weng, and Q. Zhang, "A two-stage multiobjective evolutionary algorithm for multiobjective multidepot vehicle routing problem with time windows," *IEEE Trans. Cybern.*, vol. 49, no. 7, pp. 2467–2478, Jul. 2019.

[9] L. Feng *et al.*, "Explicit evolutionary multitasking for combinatorial optimization: A case study on capacitated vehicle routing problem," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3143–3156, Jun. 2021.

[10] L. Feng *et al.*, "Solving generalized vehicle routing problem with occasional drivers via evolutionary multitasking," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3171–3184, Jun. 2021.

[11] J. Xiao, T. Zhang, J. Du, and X. Zhang, "An evolutionary multiobjective route grouping-based heuristic algorithm for large-scale capacitated vehicle routing problems," *IEEE Trans. Cybern.*, early access, Nov. 19, 2019, doi: 10.1109/TCYB.2019.2950626.

[12] K. Li, T. Zhang, and R. Wang, "Deep reinforcement learning for multiobjective optimization," *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3103–3114, Jun. 2021.

[13] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *Eur. J. Oper. Res.*, vol. 290, no. 2, pp. 405–421, 2021.

[14] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, 2015, pp. 2692–2700.

[15] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," 2016. [Online]. Available: arXiv:1611.09940.

[16] M. Nazari, A. Oroojlooy, L. Snyder, and M. Takác, "Reinforcement learning for solving the vehicle routing problem," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 31, 2018, pp. 9839–9849.

[17] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–25.

[18] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[19] A. W. Yu, D. Dohan, Q. Le, T. Luong, R. Zhao, and K. Chen, "Fast and accurate reading comprehension by combining self-attention and convolution," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–15.

[20] E. Parisotto *et al.*, "Stabilizing transformers for reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 119, 2020, pp. 7487–7498.

[21] G. Laporte, "The vehicle routing problem: An overview of exact and approximate algorithms," *Eur. J. Oper. Res.*, vol. 59, no. 3, pp. 345–358, 1992.

[22] N. Christofides, "Worst-case analysis of a new heuristic for the travelling salesman problem," Manag. Sci. Res. Group, Carnegie-Mellon Univ., Pittsburgh, PA, USA, Rep. 388, 1976.

[23] P. Moscato and C. Cotta, "A gentle introduction to memetic algorithms," in *Handbook of Metaheuristics*. Boston, MA, USA: Springer, 2003, pp. 105–144.

[24] Z. W. Geem, J. H. Kim, and G. V. Loganathan, "A new heuristic optimization algorithm: Harmony search," *Simulation*, vol. 76, no. 2, pp. 60–68, 2001.

[25] (2019). *Google: OR-Tools 7.2*. [Online]. Available: https://developers.google.com/optimization

[26] (2019). *Gurobi Optimizer Reference Manual*. [Online]. Available: http://www.gurobi.com

[27] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 53–66, Apr. 1997.

[28] S.-M. Chen and C.-Y. Chien, "Solving the traveling salesman problem based on the genetic simulated annealing ant colony system with particle swarm optimization techniques," *Expert Syst. Appl.*, vol. 38, no. 12, pp. 14439–14450, 2011.

[29] M. Zlochin, M. Birattari, N. Meuleau, and M. Dorigo, "Model-based search for combinatorial optimization: A critical survey," *Ann. Oper. Res.*, vol. 131, nos. 1–4, pp. 373–395, 2004.

[30] J. Zheng, K. He, J. Zhou, Y. Jin, and C.-M. Li, "Combining reinforcement learning with Lin-Kernighan-Helsgaun algorithm for the traveling salesman problem," 2020. [Online]. Available: arXiv:2012.04461.

[31] A. Delarue, R. Anderson, and C. Tjandraatmadja, "Reinforcement learning with combinatorial actions: An application to vehicle routing," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 609–620.

[32] S. Lee, "A cooperation online reinforcement learning approach in Ant-Q," in *Proc. Int. Conf. Neural Inf. Process. (ICONIP)*, 2006, pp. 487–494.

[33] J. P. Q. dos Santos, F. C. de Lima, R. M. Magalhaes, J. D. de Melo, and A. D. D. Neto, "A parallel hybrid implementation using genetic algorithm, grasp and reinforcement learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2009, pp. 2798–2803.

[34] F. Liu and G. Zeng, "Study of genetic algorithm with reinforcement learning to solve the TSP," *Expert Syst. Appl.*, vol. 36, no. 3, pp. 6995–7001, 2009.

[35] K. Kamei and M. Ishikawa, "Adaptive decision making in ant colony system by reinforcement learning," in *Proc. Int. Conf. Neural Inf. Process. (ICONIP)*, 2010, pp. 609–617.

[36] L. M. Gambardella and M. Dorigo, "Ant-Q: A reinforcement learning approach to the traveling salesman problem," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 1995, pp. 252–260.

[37] M. M. Alipour, S. N. Razavi, M. R. F. Derakhshi, and M. A. Balafar, "A hybrid algorithm using a genetic algorithm and multiagent reinforcement learning heuristic to solve the traveling salesman problem," *Neural Comput. Appl.*, vol. 30, no. 9, pp. 2935–2951, 2018.

[38] M. A. L. Silva, S. R. de Souza, M. J. F. Souza, and A. L. C. Bazzan, "A reinforcement learning-based multi-agent framework applied for solving routing and scheduling problems," *Expert Syst. Appl.*, vol. 131, pp. 148–171, Oct. 2019.

[39] J. J. Hopfield and D. W. Tank, "'Neural' computation of decisions in optimization problems," *Biol. Cybern.*, vol. 52, no. 3, pp. 141–152, 1985.

[40] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 3104–3112.

[41] L. Xin, W. Song, Z. Cao, and J. Zhang, "Step-wise deep learning models for solving routing problems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4861–4871, Jul. 2021.

[42] M. van Knippenberg, M. Holenderski, and V. Menkovski, "Complex vehicle routing with memory augmented neural networks," in *Proc. IEEE Conf. Ind. Cyberphys. Syst. (ICPS)*, vol. 1, 2020, pp. 303–308.

[43] K. Li, T. Zhang, R. W. Y. Wang, and Y. Han, "Deep reinforcement learning for combinatorial optimization: Covering salesman problems," 2021. [Online]. Available: arXiv:2102.05875.

[44] Q. Ma, S. Ge, D. He, D. Thaker, and I. Drori, "Combinatorial optimization by graph pointer networks and hierarchical reinforcement learning," 2019. [Online]. Available: arXiv:1911.04936.

[45] R. Zhang, A. Prokhorchuk, and J. Dauwels, "Deep reinforcement learning for traveling salesman problem with time windows and rejections," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2020, pp. 1–8.

[46] H. Wu, J. Wang, and Z. Zhang, "MODRL/D-AM: Multiobjective deep reinforcement learning algorithm using decomposition and attention model for multiobjective optimization," in *Proc. Int. Symp. Intell. Comput. Appl. (ISICA)*, 2019, pp. 575–589.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.

[47] Y. Hu, Y. Yao, and W. S. Lee, "A reinforcement learning approach for optimizing multiple traveling salesman problems over graphs," *Knowl. Based Syst.*, vol. 204, Sep. 2020, Art. no. 106244.

[48] G. Bono, J. S. Dibangoye, O. Simonin, L. Matignon, and F. Pereyron, "Solving multi-agent routing problems using deep attention mechanisms," *IEEE Trans. Intell. Transp. Syst.*, early access, Jul. 28, 2020, doi: 10.1109/TITS.2020.3009289.

[49] J. Zhao, M. Mao, X. Zhao, and J. Zou, "A hybrid of deep reinforcement learning and local search for the vehicle routing problems," *IEEE Trans. Intell. Transp. Syst.*, early access, Jul. 15, 2020, doi: 10.1109/TITS.2020.3003163.

[50] M. Deudon, P. Cournut, A. Lacoste, Y. Adulyasak, and L.-M. Rousseau, "Learning heuristics for the TSP by policy gradient," in *Proc. Int. Conf. Integr. Constraint Program. Artif. Intell. Oper. Res.(CPAIOR)*, 2018, pp. 170–181.

[51] Y. Kaempfer and L. Wolf, "Learning the multiple traveling salesmen problem with permutation invariant pooling networks," 2018. [Online]. Available: arXiv:1803.09621.

[52] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist. Human Lang. Technol. (NAACL-HLT)*, vol. 1, 2019, pp. 4171–4186.

[53] Y. Xu, L. Chen, M. Fang, Y. Wang, and C. Zhang, "Deep reinforcement learning with transformers for text adventure games," in *Proc. IEEE Conf. Games (CoG)*, 2020, pp. 65–72.

[54] H. Dai, E. B. Khalil, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 6351–6361.

[55] A. Nowak, S. Villar, A. S. Bandeira, and J. Bruna, "A note on learning algorithms for quadratic assignment with graph neural networks," 2017. [Online]. Available: arXiv:1706.07450v1.

[56] C. K. Joshi, T. Laurent, and X. Bresson, "An efficient graph convolutional network technique for the travelling salesman problem," 2019. [Online]. Available: arXiv:1906.01227.

[57] L. Duan *et al.*, "Efficiently solving the practical vehicle routing problem: A novel joint learning approach," in *Proc. ACM SIGKDD Int. Conf. Knowl. Disc. Data Min. (KDD)*, 2020, pp. 3054–3063.

[58] V. Mnih, K. Kavukcuoglu, D. Silver, A. G. I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," 2013. [Online]. Available: arXiv:1312.5602.

[59] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[60] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, p. 12.

[61] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[62] X. Chen and Y. Tian, "Learning to perform local rewriting for combinatorial optimization," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 6278–6289.

[63] H. Lu, X. Zhang, and S. Yang, "A learning-based iterative method for solving vehicle routing problems," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–15.

[64] Y. Wu, W. Song, Z. Cao, J. Zhang, and A. Lim, "Learning improvement heuristics for solving the travelling salesman problem," 2019. [Online]. Available: arXiv:1912.05784.

[65] L. Gao, M. Chen, Q. Chen, G. Luo, N. Zhu, and Z. Liu, "Learn to design the heuristics for vehicle routing problem," 2020. [Online]. Available: arXiv:2002.08539.

[66] P. R. d. O. da Costa, J. Rhuggenaath, Y. Zhang, and A. Akcay, "Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning," in *Proc. Asian Conf. Mach. Learn. (ACML)*, vol. 129, 2020, pp. 465–480.

[67] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min, "POMO: Policy optimization with multiple optima for reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 21188–21198.

[68] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 37, 2015, pp. 448–456.

[69] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Highway networks," 2015. [Online]. Available: arXiv:1505.00387.

[70] C. Cangea, P. Veličković, N. Jovanović, T. Kipf, and P. Liò, "Towards sparse hierarchical graph classifiers," 2018. [Online]. Available: arXiv:1811.01287.

[71] D. Bahdanau, K. H. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, pp. 1–15.

[72] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 229–256, 1992.

[73] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 12, 2000, pp. 1057–1063.

[74] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, pp. 1–15.

[75] M. Dror and P. Trudeau, "Split delivery routing," *Naval Res. Logist.*, vol. 37, no. 3, pp. 383–402, 1990.

[76] B. L. Golden, L. Levy, and R. Vohra, "The orienteering problem," *Naval Res. Logist.*, vol. 34, no. 3, pp. 307–318, 1987.

[77] M. Fischetti, J. J. S. González, and P. Toth, "Solving the orienteering problem through branch-and-cut," *INFORMS J. Comput.*, vol. 10, no. 2, pp. 133–148, 1998.

[78] E. Balas, "The prize collecting traveling salesman problem," *Networks*, vol. 19, no. 6, pp. 621–636, 1989.

[79] K. A. De Jong, 'An analysis of the behavior of a class of genetic adaptive systems," Ph.D. dissertation, Dept. Comput. Commun. Sci., Univ. Michigan, Ann Arbor, MI, USA, 1975.

[80] W. Khatib and P. J. Fleming, "The stud GA: A mini revolution?" in *Proc. Int. Conf. Parallel Problem Solving Nat. (PPSN)*, 1998, pp. 683–691.

[81] K. Xu *et al.*, "Show, attend and tell: Neural image caption generation with visual attention," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 37, 2015, pp. 2048–2057.

[82] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[83] S. Greydanus, A. Koul, J. Dodge, and A. Fern, "Visualizing and understanding Atari agents," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 80, 2018, pp. 1792–1801.

[84] A. Mott, D. Zoran, M. Chrzanowski, D. Wierstra, and D. J. Rezende, "Towards interpretable reinforcement learning using attention augmented agents," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 12329–12338.

[85] Z.-H. Fu, K.-B. Qiu, and H. Zha, "Generalize a small pre-trained model to arbitrarily large TSP instances," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 35, 2021, pp. 7474–7482.

[86] Z. Xing and S. Tu, "A graph neural network assisted Monte Carlo tree search approach to traveling salesman problem," *IEEE Access*, vol. 8, pp. 108418–108428, 2020.

[87] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 70, 2017, pp. 1126–1135.

[88] M. Laskin, K. Lee, A. Stooke, L. Pinto, P. Abbeel, and A. Srinivas, "Reinforcement learning with augmented data," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 19884–19895.

[89] Y. Xu, M. Fang, L. Chen, Y. Du, J. T. Zhou, and C. Zhang, "Deep reinforcement learning with stacked hierarchical attention for text-based games," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 16495–16507.

[90] C.-X. Wu, M.-H. Liao, M. Karatas, S.-Y. Chen, and Y.-J. Zheng, "Real-time neural network scheduling of emergency medical mask production during COVID-19," *Appl. Soft Comput.*, vol. 97, Dec. 2020, Art. no. 106790.

[91] Y. Shao, A. Rezaee, S. C. Liew, and V. W. S. Chan, "Significant sampling for shortest path routing: A deep reinforcement learning solution," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2234–2248, Oct. 2020.

[92] M. Fang, Y. Li, and T. Cohn, "Learning how to active learn: A deep reinforcement learning approach," in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2017, pp. 595–605, doi: 10.18653/v1/D17-1063.

[93] M. Fang, C. Zhou, B. Shi, B. Gong, J. Xu, and T. Zhang, "DHER: Hindsight experience replay for dynamic goals," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[94] M. Fang, T. Zhou, Y. Du, L. Han, and Z. Zhang, "Curriculum-guided hindsight experience replay," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019.

[95] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: arXiv:1707.06347.

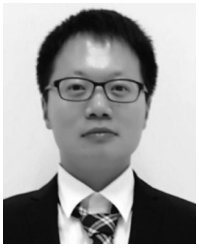
[96] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.



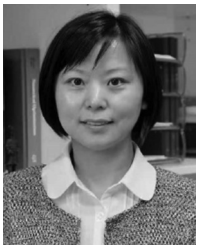
Yunqiu Xu (Graduate Student Member, IEEE) received the B.S. degree in polymer materials from Zhejiang University, Hangzhou, China, in 2014, and the M.S. degree in information technology from the University of New South Wales, Sydney, NSW, Australia, in 2018. He is currently pursuing the Ph.D. degree in computer science with the University of Technology Sydney, Sydney.

His research focuses on deep learning, reinforcement learning, natural language processing, and intelligent transportation systems.



Meng Fang received the Ph.D. degree from the University of Technology, Sydney, NSW, Australia, in 2015.

He was a Research Fellow with the School of Computing and Information Systems, University of Melbourne, Melbourne, VIC, Australia. He is currently an Assistant Professor with the Department of Mathematics and Computer Science, Eindhoven University of Technology, Eindhoven, The Netherlands. His research focuses on reinforcement learning and natural language processing.



Ling Chen (Senior Member, IEEE) received the Ph.D. degree from Nanyang Technological University, Singapore, in 2008.

She was a Postdoctoral Research Fellow with the L3S Research Center, University of Hannover, Hanover, Germany. She is an Associate Professor with the Centre for Artificial Intelligence, University of Technology Sydney, Sydney, NSW, Australia. Her papers appear in major conferences and journals, including SIGKDD, ICDM, SDM, and ACM TOIS. Her research interests include data mining and machine learning, social network analysis, and recommender systems.



Gangyan Xu (Member, IEEE) received the B.S. degree in automation and the M.E. degree in systems engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2009 and 2012, respectively, and the Ph.D. degree in systems engineering from The University of Hong Kong, Hong Kong, in 2016.

He is an Assistant Professor with the Harbin Institute of Technology, Shenzhen, China. He has worked as a Research Fellow and a Research Assistant with Nanyang Technological University, Singapore, and the City University of Hong Kong, Hong Kong. His research interests include data-driven optimization, intelligent transportation management and control, resilient systems, and emergency management.



Yali Du received the Ph.D. degree from the University of Technology Sydney, Sydney, NSW, Australia, in 2019.

She is a Postdoctoral Research Fellow with the University College London, London, U.K. Her research output has been widely published in prestigious venues, including ICML, NeurIPS, and IEEE TNNLS. She is generally interested in machine learning, reinforcement learning and their broad applications in data science, game AI, and classic control and decision-making tasks.



Chengqi Zhang (Senior Member, IEEE) received the Ph.D. degree from the University of Queensland Brisbane, Brisbane, QLD, Australia, in 1991, and the D.Sc. degree (Higher Doctorate) from Deakin University, Geelong, VIC, Australia, in 2002.

Since 2001, he has been a Professor of Information Technology with the University of Technology Sydney (UTS), Sydney, NSW, Australia, where he has been the Executive Director of UTS Data Science since 2016. Since 2005, he has been the Chairman of the Australian Computer Society National Committee for Artificial Intelligence. He has published more than 200 research papers, including several in first-class international journals, such as *Artificial Intelligence*, and the IEEE and ACM TRANSACTIONS. He has published six monographs and edited 16 books, and has attracted 11 Australian Research Council grants. His research interests mainly focus on data mining and its applications.

Dr. Zhang has served as an Associate Editor for three international journals, including the IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING from 2005 to 2008; and as a General Chair, a PC Chair, or an Organizing Chair for five international conferences, including ICDM 2010 and WI/IAT 2008. He is the General Co-Chair of KDD 2015 in Sydney, the Local Arrangements Chair of IJCAI-2017 in Melbourne, and a Fellow of the Australian Computer Society.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:37:51 UTC from IEEE Xplore. Restrictions apply.